

Импорты

import string

Доступ к таблице символов пунктуации для удаления. Предоставляет `string.punctuation` — строку со всеми знаками препинания.

import nltk

Библиотека для обработки естественного языка (NLP). Используется для загрузки `stopwords` и токенизатора `punkt`.

import pymorphy2

Морфологический анализатор для русского языка. Лемматизация слов (приведение к начальной форме).

from razdel import tokenize

Токенизатор для русского языка. Разбивает текст на слова с учетом особенностей русского языка.

from nltk.corpus import stopwords

Загрузка предустановленных стоп-слов. Хотя используется кастомный список, NLTK готов к расширению.

Функция предобработки текста

tclean(text)

Полная очистка и нормализация русского текста для машинного обучения.

1. **russian_stopwords = []**
Инициализация списка стоп-слов. Создает пустой список для хранения стоп-слов из файла.
2. **file = open("Back/stop-ru.txt", "r")**
Загрузка пользовательского списка стоп-слов. Открывает файл `stop-ru.txt` для чтения.
3. **for i in file: russian_stopwords.append(i[:-1])**
Чтение стоп-слов построчно. Добавляет каждую строку файла (без символа переноса строки `\n`) в список.
4. **morph = pymorphy2.MorphAnalyzer()**
Инициализация морфологического анализатора. Создает объект для лемматизации русских слов.
5. **text = text.lower()**
Приведение текста к нижнему регистру. Убирает различия между заглавными и строчными буквами.
6. **translator = str.maketrans("", "", string.punctuation + '0123456789')**
Создание таблицы перевода для удаления символов. Генерирует таблицу, где пунктуация + цифры → пустота.
7. **text_cleaned = text.translate(translator)**
Удаление пунктуации и цифр из текста. Применяет таблицу перевода к тексту.
8. **tokens = [token.text for token in tokenize(text_cleaned)]**
Токенизация очищенного текста. Разбивает текст на отдельные слова с помощью `razdel.tokenize()`.

9. **tokens = [t for t in tokens if t not in russian_stopwords]**
Фильтрация стоп-слов. Оставляет только токены, отсутствующие в списке стоп-слов.
10. **lemmas = .normal_form for t in tokens]**
Лемматизация токенов. Для каждого токена берет первую морфологическую форму (нормальную форму).
11. **return lemmas**
Возврат результата обработки. Возвращает список лемм (начальных форм слов).

Использование в системе:

Результат `tclean()` передается в `TfidfVectorizer` класса `ML` для создания векторов текстов книг.

Документация модуля ML

Глобальные настройки

****# SKLEARN_ALLOW_DEPRECATED_SKLEARNACKAGE_INSTALL=True**
Разрешить установку устаревших версий scikit-learn. Переменная окружения для совместимости пакетов.

Импорты

import pandas as pd
Табличные данные книг. `DataFrame` для библиотеки.

import sqlite3
База данных SQLite. Подключение к `DataBase.db`.

import csv
Чтение CSV файлов. Парсинг `Combined_clean_tables.csv`.

from sklearn.feature_extraction.text import TfidfVectorizer
TF-IDF векторизация. Текст → числовые векторы.

from sklearn.metrics.pairwise import cosine_similarity
Поиск похожих книг. Косинусное сходство векторов.

from Data.book_for_tgBot import Book
Объекты книг. `Book.byID()` для рекомендаций.

from .model import LLM
Языковая модель. `LLM.describeGenres()`.

Функция add_user_to_db

def add_user_to_db(userId):
Регистрация пользователя. Добавляет `userId` в таблицу `users`.

db = sqlite3.connect("Data/DataBase.db")

Подключение к БД. Открывает базу данных.

cursor = db.cursor()

SQL запросы. Создает курсор для выполнения команд.

cursor.execute("PRAGMA foreign_keys = ON;")

Каскадное удаление. Включает внешние ключи.

cursor.execute("SELECT id FROM users WHERE id = ?", (userId,))

Проверка существования. Ищет пользователя по id.

exists = cursor.fetchone()

Результат проверки. Получает первую строку результата.

if exists: db.close()

Закрытие при существовании. Завершает если пользователь есть.

else: cursor.execute("INSERT INTO users (id) VALUES (?)", (userId,))

Добавление нового. INSERT нового пользователя.

db.commit()

Сохранение изменений. Фиксирует транзакцию.

db.close()

Освобождение ресурсов. Закрывает соединение.

Функция `get_list_reviews`

def get_list_reviews(userId, review: int):

Список книг по оценке. Возвращает ID по типу отзыва (0-3).

db = sqlite3.connect("Data/DataBase.db")

Подключение БД. Открывает базу.

cursor = db.cursor()

SQL курсор. Создает курсор.

cursor.execute("PRAGMA foreign_keys = ON;")

Внешние ключи. Включает каскады.

**cursor.execute("SELECT idBook FROM review WHERE idUser = ? AND esteem = ?",
(userId, review))**

Поиск отзывов. SELECT по пользователю и типу оценки.

b = cursor.fetchall()

Все результаты. Получает все строки.

db.close()

Закрытие соединения. Освобождает ресурсы.

return for r in b]

Список ID. Извлекает idBook из каждой строки.

Класс ML init(self)

db = sqlite3.connect("Data/DataBase.db")

База данных. Подключение.

cursor = db.cursor()

SQL операции. Курсор.

cursor.execute("PRAGMA foreign_keys = ON;")

Каскады. Внешние ключи.

cursor.executescript(open("Data/creation.sql", 'r').read())

Создание таблиц. Выполняет SQL скрипт создания.

if cursor.execute("SELECT * FROM books WHERE id = ?", (1,)).fetchone() == None:

Проверка данных. Есть ли хотя бы 1 книга.

tuples = []

Список для INSERT. Пустой список кортежей.

with open("Data/Combined_clean_tables.csv", 'r', encoding="utf-8") as file:

Чтение CSV. Открывает файл книг.

reader = csv.DictReader(file, delimiter=';')

Парсинг CSV. Словарь по заголовкам.

for row in reader:

Обработка строк. Итерация по книгам.

tuples.append((row["name"], row["author"], ...))

Подготовка данных. Кортеж для SQL.

cursor.executemany("INSERT INTO books ...", tuples)

Массовое добавление. Вставляет все книги.

db.commit()

Сохранение. Фиксирует изменения.

print("Books have been added to db from csv")

Лог. Сообщение об успехе.

self.df = pd.read_sql_query("SELECT * FROM books", db)

Pandas таблица. Загружает все книги в DataFrame.

self.llm = LLM()

Языковая модель. Инициализация LLM.

def prepare_text(row):

Текст для TF-IDF. Внутренняя функция.

```
author = row['author'] or ""
```

Автор или пусто. Безопасное извлечение.

```
genres = (row['genre'] + ", ") * 3
```

Увеличение веса жанров. Повторяет жанры 3 раза.

```
return f"{author} {genres} {description}".lower()
```

Объединенный текст. Конкатенация в нижний регистр.

```
self.df['text'] = self.df.apply(prepare_text, axis=1)
```

Добавление столбца. Применяет к каждой строке.

```
with open('Back/stop-ru.txt') as f: sw = f.read().splitlines()
```

Стоп-слова. Загружает из файла.

```
self.vectorizer = TfidfVectorizer(stop_words=sw, max_features=20000)
```

TF-IDF модель. Создает векторизатор.

```
self.tfidf_matrix = self.vectorizer.fit_transform(self.df['text'])
```

Векторы текстов. Преобразует все тексты.

```
db.close()
```

Закрытие БД. Освобождает соединение.

NameCossim(self, BookName: str)

```
def recommend_similar(book_name, top_n=5):
```

Внутренний поиск похожих. Локальная функция.

```
mask = self.df['name'].str.lower() == book_name.lower()
```

Фильтр по названию. Булева маска.

```
if not mask.any(): return None
```

Проверка существования. Возврат если книги нет.

```
idx = self.df[mask].index
```

Индекс книги. Первый найденный индекс.

```
cos_sim = cosine_similarity(self.tfidf_matrix[idx], self.tfidf_matrix).flatten()
```

Сходство с книгой. Косинус всех векторов.

```
similar_idxs = cos_sim.argsort()[::-1]
```

Сортировка по убыванию. Индексы от большего к меньшему.

```
similar_idxs = similar_idxs[similar_idxs != idx][:top_n]
```

Исключить саму книгу. Топ-N без оригинала.

```
return self.df.iloc[similar_idxs][['name', 'author', 'genre', 'id', 'picPath']]
```

Информация о книгах. Выборка столбцов.

```
books = recommend_similar(BookName, top_n=100)['id'].tolist()
```

Список ID. Извлекает 100 ID.

return books

Результат метода. Возврат списка ID.

GenreCossim(self, user_input: str)

Genre = self.llm.describeGenres(user_input)

LLM анализ жанров. Генерирует жанры из запроса.

def recommend_similar(genre, top_n=5):

Поиск по жанрам. Внутренняя функция.

genres_list = self.df['genre'].tolist()

Все жанры БД. Список жанров книг.

vectorizer = TfidfVectorizer().fit(genres_list + [genre])

Векторизатор жанров. Обучает на жанрах + запрос.

tfidf_matrix = vectorizer.transform(genres_list + [genre])

Матрица TF-IDF. Преобразует все жанры.

cos_sim = cosine_similarity(tfidf_matrix[-1], tfidf_matrix[:-1]).flatten()

Сходство запроса. Последний вектор(запрос) vs остальные.

top_idx = cos_sim.argsort()[::-1][:top_n]

Топ похожих. Сортировка по сходству.

books = recommend_similar(Genre, top_n=100)['id'].tolist()

ID рекомендаций. 100 лучших по жанрам

Likes(self, UserId: int)

db = sqlite3.connect("Data/DataBase.db")

БД отзывов. Подключение.

cursor.execute("SELECT * FROM review WHERE idUser = ? AND esteem = ?", (UserId, 1))

Лайки пользователя. SELECT esteem=1.

likes = get_list_reviews(UserId, 1)

Список лайкнутых ID. Вызов функции.

lists = []

Аккумулятор рекомендаций. Пустой список.

for id in likes: theBook = Book.byID(id)

Книга по ID. Создает Book объект.

lists += self.NameCossim(theBook.name)

Похожие на лайк. Добавляет рекомендации.

return list(set(lists))

Уникальные. Удаляет дубликаты.

recommendations(self, UserId: int)

default_recommenations = [217, 23099, ...]

Базовые рекомендации. Хардкод топ книг.

likes/dislikes/saved/seen = get_list_reviews(UserId, 1/2/3/0)

История пользователя. Все типы отзывов.

booksId = self.Likes(UserId)

Рекомендации по лайкам. Вызов Likes().

booksId = list(set(default_recommenations + booksId))

Объединение. Уникальный список.

for i in range(len(booksId)):

Фильтрация. Проверка каждой книги.

if booksId[i] in dislikes/saved/seen/likes: continue

Исключения. Пропускает нежелательные.

result.append(booksId[i])

Подходящие. Добавляет в результат.

search(bookName: str, ...)

db = sqlite3.connect(file_path)

Поисковая БД. Подключение.

cursor.execute("SELECT id, name FROM books")

Все книги. Загружает id+name.

query = bookName.strip().lower()

Нормализация запроса. Нижний регистр.

for row in rows:

Линейный поиск. Проверяет каждую книгу.

if query in book_name.strip().lower():

Подстрока. LIKE '%query%'.

books_found.append(book_id)

Результаты. Добавляет ID.

if len(books_found) >= limit: break

Ограничение. Прерывает на limit.

llm_recommendations(self, UserId, UserInput)

dislikes/saved/seen = get_list_reviews(UserId, 2/3/0)

Негативная история. Получает списки.

books = self.GenreCossim(UserInput)

LLM рекомендации. Жанровый поиск.

for i in books:

Фильтрация. Проверка каждого ID.

if i in dislikes/saved/seen: books.pop(i)

Удаление нежелательных. Убирает из списка.

return books

Очищенный список. Возврат результата.

ml_object.py

from Back.ml import ML

Импорт класса. Доступ к ML.

ml = ML()

Глобальный singleton. Создает единственный экземпляр.

Глобальные настройки

****# SKLEARN_ALLOW_DEPRECATED_SKLEARN_PACKAGEINSTALL=True**

Разрешает установку устаревших версий пакета scikit-learn, необходима для обеспечения совместимости с различными версиями зависимостей при развертывании системы рекомендаций. Устанавливает переменную окружения, которая подавляет предупреждения об устаревших пакетах и гарантирует успешную установку всех необходимых ML-библиотек.

Импорты

import pandas as pd

Обеспечивает работу с табличными данными книг в формате DataFrame, используется для загрузки, фильтрации и преобразования библиотечного каталога из базы данных и CSV-файлов в удобную структуру для ML-обработки.

import sqlite3

Предоставляет подключение к локальной SQLite базе данных DataBase.db, где хранятся пользователи, книги и отзывы, обеспечивая постоянное хранение данных рекомендационной системы.

import csv

Необходим для чтения и парсинга CSV-файлов с данными книг (Combined_clean_tables.csv), содержащих информацию о названиях, авторах, жанрах и обложках, полученную из парсера litres.ru.

from sklearn.feature_extraction.text import TfidfVectorizer

Реализует TF-IDF векторизацию текстовых описаний книг (автор + жанры ×3 + описание),

преобразует неструктурированный текст в числовые векторы фиксированной размерности (max_features=20000) для последующего расчета сходства.

from sklearn.metrics.pairwise import cosine_similarity

Вычисляет косинусное сходство между TF-IDF векторами книг, основной метрикой для поиска похожих рекомендаций по названию, жанрам и пользовательским предпочтениям

from Data.book_for_tgBot import Book

Импортирует класс Book для создания объектов книг по ID из базы данных, используется при формировании рекомендаций для получения полной информации о книге (название, автор, жанры, обложка).

from .model import LLM

Подключает языковую модель LLM для генерации описаний жанров из пользовательских запросов, расширяет функциональность поиска за счет семантического понимания предпочтений ("хочу детектив как у Агаты Кристи").

Функция add_user_to_db

def add_user_to_db(userId):

Регистрирует нового пользователя Telegram в таблице users по уникальному userId, создавая запись только при первом обращении к боту. Проверяет существование через SELECT, выполняет INSERT при необходимости, включает foreign_keys для каскадного удаления связанных отзывов.

db = sqlite3.connect("Data/DataBase.db")

Устанавливает соединение с основной базой данных проекта, где хранятся все пользователи, книги и их оценки для персонализации рекомендаций.

cursor = db.cursor()

Создает курсор для выполнения SQL-команд, необходим для всех операций чтения/записи в базу данных пользователей и отзывов.

cursor.execute("PRAGMA foreign_keys = ON;")

Активирует поддержку внешних ключей SQLite с каскадным удалением, гарантируя целостность связей между таблицами users, books и review при удалении записей.

cursor.execute("SELECT id FROM users WHERE id = ?", (userId,))

Проверяет существование пользователя по Telegram userId, используя параметризованный запрос для предотвращения SQL-инъекций.

exists = cursor.fetchone()

Получает результат проверки (первую строку или None), определяет необходимость создания новой записи пользователя.

if exists: db.close()

При наличии пользователя завершает транзакцию и закрывает соединение, экономя ресурсы базы данных.

else: cursor.execute("INSERT INTO users (id) VALUES (?)", (userId,))

Создает запись нового пользователя в таблицу users с Telegram ID как первичным ключом

db.commit()

Фиксирует изменения в базе данных, делая нового пользователя доступным для системы рекомендаций.

db.close()

Освобождает ресурсы базы данных, закрывая соединение после завершения операции регистрации.

Функция get_list_reviews

def get_list_reviews(userId, review: int):

Возвращает список ID книг по конкретному типу оценки пользователя (0=просмотрено, 1=лайк, 2=дизлайк, 3=сохранено), используется для персонализации рекомендаций и фильтрации нежелательного контента.

db = sqlite3.connect("Data/DataBase.db")

Открывает соединение с базой данных отзывов для анализа истории взаимодействий конкретного пользователя.

cursor = db.cursor()

Создает курсор для выполнения SELECT-запроса по таблице review.

cursor.execute("PRAGMA foreign_keys = ON;")

Включает внешние ключи для обеспечения целостности данных при работе с отзывами.

cursor.execute("SELECT idBook FROM review WHERE idUser = ? AND esteem = ?", (userId, review))

Извлекает ID книг с заданным типом оценки для конкретного пользователя, используя безопасные параметры.

b = cursor.fetchall()

Получает все строки результата запроса в виде списка кортежей для дальнейшей обработки.

db.close()

Закрывает соединение с базой данных после извлечения истории отзывов.

return [r for r in b]

Преобразует кортежи результатов в чистый список ID книг, готовый для рекомендационного движка.

Класс ML init(self)

db = sqlite3.connect("Data/DataBase.db")

Устанавливает соединение с основной базой данных системы рекомендаций, где хранятся все таблицы (users, books, review) для инициализации ML-модели.

cursor = db.cursor()

Создает курсор SQLite для выполнения команд создания таблиц, проверки данных и массовой загрузки книг из CSV

cursor.execute("PRAGMA foreign_keys = ON;")

Активирует внешние ключи с каскадным удалением для обеспечения референциальной целостности между таблицами при инициализации базы.

cursor.executescript(open("Data/creation.sql", 'r').read())

Выполняет полный SQL-скрипт создания схемы базы данных (таблицы users, books, review с внешними ключами), гарантируя наличие всех необходимых структур.

if cursor.execute("SELECT * FROM books WHERE id = ?", (1,)).fetchone() == None:

Проверяет наличие хотя бы одной книги в таблице books (по ID=1), определяет необходимость начальной загрузки данных из CSV.

tuples = []

Инициализирует пустой список кортежей для массовой вставки данных книг, оптимизируя производительность загрузки тысяч записей.

with open("Data/Combined_clean_tables.csv", 'r', encoding="utf-8") as file:

Открывает основной CSV-файл с данными книг (парсинг litres.ru), используя UTF-8 для корректной обработки русских названий и описаний.

reader = csv.DictReader(file, delimiter=';')

Создает итератор по строкам CSV как словари (ключ=заголовок), парсит данные книг с разделителем точка с запятой.

for row in reader:

Проходит по всем строкам CSV-файла книг, извлекая данные для каждой книги (название, автор, жанры, описание, обложка, рейтинг).

tuples.append((row["name"], row["author"], ...))

Формирует кортеж данных каждой книги для SQL INSERT, извлекая все необходимые поля из словаря CSV-строки.

cursor.executemany("INSERT INTO books ...", tuples)

Выполняет массовую вставку всех книг из CSV в таблицу books одним запросом, значительно ускоряя инициализацию базы (тысячи строк).

db.commit()

Фиксирует все изменения массовой вставки в базе данных, делая библиотеку книг доступной для рекомендаций.

print("Books have been added to db from csv")

Выводит лог-сообщение об успешной загрузке книг из CSV в базу данных для мониторинга процесса инициализации

self.df = pd.read_sql_query("SELECT * FROM books", db)

Загружает всю таблицу books в Pandas DataFrame для быстрого доступа и ML-обработки всех книг в памяти.

self.llm = LLM()

Инициализирует языковую модель Qwen3-4B-Instruct для генерации жанровых описаний из пользовательских запросов.

def prepare_text(row):

Определяет внутреннюю функцию для создания объединенного текстового представления книги для TF-IDF векторизации.

author = row['author'] or ""

Безопасно извлекает имя автора из строки DataFrame, заменяя None/NaN на пустую строку для избежания ошибок конкатенации.

genres = (row['genre'] + ", ") * 3

Увеличивает вес жанров путем тройного повторения (genre + ", "), делая их доминирующим фактором в TF-IDF векторах.

return f"{author} {genres} {description}".lower()

Конкатенирует автор + жанры×3 + описание в нижнем регистре, формируя единый текст для векторизации каждой книги.

self.df['text'] = self.df.apply(prepare_text, axis=1)

Применяет функцию prepare_text ко всем строкам DataFrame, создавая новый столбец 'text' для TF-IDF обработки.

with open('Back/stop-ru.txt') as f: sw = f.read().splitlines()

Загружает кастомный список русских стоп-слов из файла для исключения из TF-IDF векторизации (не считать "и", "в", "на").

self.vectorizer = TfidfVectorizer(stop_words=sw, max_features=20000)

Создает TF-IDF векторизатор с русскими стоп-словами и ограничением словаря до 20k наиболее значимых терминов

self.tfidf_matrix = self.vectorizer.fit_transform(self.df['text'])

Преобразует все текстовые описания книг в разреженную матрицу TF-IDF векторов, готовую для расчета косинусного сходства.

db.close()

Закрывает соединение с базой данных после полной инициализации ML-модели, освобождая системные ресурсы.

NameCossim(self, BookName: str)

def recommend_similar(book_name, top_n=5):

Внутренняя вспомогательная функция для поиска похожих книг по TF-IDF сходству, используется для получения топ-N рекомендаций по названию книги с последующей фильтрацией и выборкой данных.

mask = self.df['name'].str.lower() == book_name.lower()

Создает булеву маску DataFrame для точного поиска книги по названию в нижнем регистре, игнорируя регистр при сравнении строк.

if not mask.any(): return None

Проверяет наличие книги в библиотеке, возвращает None если название не найдено, предотвращая ошибки при работе с несуществующими книгами.

idx = self.df[mask].index

Получает индекс первой (единственной) найденной книги в DataFrame для использования в расчете косинусного сходства с остальной библиотекой.

cos_sim = cosine_similarity(self.tfidf_matrix[idx], self.tfidf_matrix).flatten()

Вычисляет косинусное сходство TF-IDF вектора целевой книги со всеми книгами библиотеки, возвращая одномерный массив оценок близости.

similar_idxs = cos_sim.argsort()[::-1]

Сортирует индексы книг по убыванию косинусного сходства (от наиболее похожих к наименее), используя argsort для получения ранжированных индексов.

similar_idxs = similar_idxs[similar_idxs != idx][:top_n]

Исключает индекс самой книги из результатов и ограничивает топ-N наиболее похожих, предотвращая саморекомендацию.

return self.df.iloc[similar_idxs][['name', 'author', 'genre', 'id', 'picPath']]

Возвращает DataFrame с информацией о топ-N похожих книгах (название, автор, жанры, ID, обложка) для дальнейшей обработки.

books = recommend_similar(BookName, top_n=100)['id'].tolist()

Вызывает внутренний поиск для 100 похожих книг и извлекает только их ID в виде списка для основной логики рекомендаций.

return books

Возвращает финальный список ID рекомендованных книг, готовый для фильтрации по истории пользователя и отправки в Telegram-бот.

GenreCossim(self, user_input: str)

Genre = self.llm.describeGenres(user_input)

Использует языковую модель LLM для преобразования пользовательского запроса (например, "хочу фантастику как у Стругацких") в структурированное описание жанров ("фантастика, философская фантастика").

def recommend_similar(genre, top_n=5):

Внутренняя функция для поиска книг по сходству жанров, обучает отдельный TF-IDF векторизатор только на жанрах (без полного текста книги).

genres_list = self.df['genre'].tolist()

Извлекает все жанры из библиотеки книг в виде плоского списка строк для обучения векторизатора жанрового поиска.

vectorizer = TfidfVectorizer().fit(genres_list + [genre])

Создает и обучает новый TF-IDF векторизатор на всех жанрах библиотеки + запросе пользователя, адаптируясь к конкретному поиску.

```
tfidf_matrix = vectorizer.transform(genres_list + [genre])
```

Преобразует все жанры библиотеки и пользовательский запрос в TF-IDF векторы с общим словарем, последний вектор соответствует запросу.

```
cos_sim = cosine_similarity(tfidf_matrix[-1], tfidf_matrix[:-1]).flatten()
```

Вычисляет косинусное сходство запроса (последний вектор) со всеми жанрами книг, исключая сам запрос из сравнения.

```
top_idxs = cos_sim.argsort()[::-1][:top_n]
```

Получает индексы топ-N книг с наиболее похожими жанрами по убыванию TF-IDF сходства.

```
books = recommend_similar(Genre, top_n=100)['id'].tolist()
```

Применяет жанровый поиск к LLM-описанию, возвращая 100 ID книг с наиболее подходящими жанрами для дальнейшей фильтрации.

Likes(self, UserId: int)

```
db = sqlite3.connect("Data/DataBase.db")
```

Открывает соединение с базой данных для извлечения истории лайков конкретного пользователя, использует таблицу review с esteem=1.

```
cursor.execute("SELECT * FROM review WHERE idUser = ? AND esteem = ?", (UserId, 1))
```

Выполняет запрос для получения всех лайкнуемых книг пользователя (оценка 1), загружает полную информацию об отзывах.

```
likes = get_list_reviews(UserId, 1)
```

Вызывает вспомогательную функцию для получения чистого списка ID лайкнуемых книг, оптимизируя повторный SQL-запрос.

```
lists = []
```

Инициализирует пустой аккумулятор для сбора рекомендаций по всем лайкнутым книгам пользователя.

```
for id in likes: theBook = Book.byID(id)
```

Проходит по всем лайкнутым книгам, создавая объект Book по ID для доступа к названию и другим метаданным.

```
lists += self.NameCossim(theBook.name)
```

Для каждой лайкнутой книги находит 100 похожих по TF-IDF и добавляет их ID в общий список рекомендаций.

```
return list(set(lists))
```

Удаляет дубликаты из объединенного списка рекомендаций, возвращая уникальные ID похожих на лайкнутые книги.

recommendations(self, UserId: int)

default_reccomendations = [217, 23099, ...]

Хардкодированный список популярных книг (топ-выбор по популярности/рейтингу), используется как базовые рекомендации для новых пользователей.

likes/dislikes/saved/seen = get_list_reviews(UserId, 1/2/3/0)

Загружает полную историю взаимодействий пользователя: лайки(1), дизлайки(2), сохраненные(3), просмотренные(0) для персонализации.

booksId = self.Likes(UserId)

Получает рекомендации на основе лайкнутых книг пользователя, используя алгоритм поиска похожих по названию.

booksId = list(set(default_reccomendations + booksId))

Объединяет базовые рекомендации с персонализированными, удаляя дубликаты для создания расширенного пула кандидатов.

for i in range(len(booksId)):

Последовательно проверяет каждую книгу из объединенного списка на соответствие критериям фильтрации.

if booksId[i] in dislikes/saved/seen/likes: continue

Исключает книги из негативной истории (дизлайки), уже просмотренные/сохраненные/лайкнутые, фокусируясь на новом контенте.

result.append(booksId[i])

Добавляет в финальный результат только подходящие книги, прошедшие все фильтры персонализации.

search(bookName: str, ...) (staticmethod)

db = sqlite3.connect(file_path)

Подключается к базе данных книг для выполнения полнотекстового поиска по названию, поддерживает произвольный путь к БД.

cursor.execute("SELECT id, name FROM books")

Загружает все ID и названия книг в память для быстрого линейного поиска (альтернатива SQL LIKE для больших баз).

query = bookName.strip().lower()

Нормализует поисковый запрос: удаляет пробелы по краям и приводит к нижнему регистру для регистронезависимого поиска.

for row in rows:

Выполняет линейный поиск по всем книгам, проверяя каждое название на наличие подстроки запроса.

if query in book_name.strip().lower():

Проверяет вхождение поискового запроса как подстроки в названии книги (аналог SQL LIKE '%query%').

books_found.append(book_id)

Добавляет ID найденной книги в результат поиска для возврата пользователю.

if len(books_found) >= limit: break

Прерывает поиск при достижении лимита результатов, оптимизируя производительность для популярных запросов.

llm_recommendations(self, UserId, UserInput)

dislikes/saved/seen = get_list_reviews(UserId, 2/3/0)

Загружает негативную историю пользователя (дизлайки, сохраненные, просмотренные) для исключения из LLM-рекомендаций.

books = self.GenreCossim(UserInput)

Получает рекомендации по жанрам, сгенерированным LLM из пользовательского запроса (например, "триллер психологический").

for i in books:

Проходит по всем ID книг из жанрового поиска для применения персональных фильтров.

if i in dislikes/saved/seen: books.pop(i)

Удаляет из списка книги из негативной истории пользователя, обеспечивая релевантность рекомендаций.

return books

Возвращает очищенный список LLM-рекомендаций, адаптированный под предпочтения пользователя.

ml_object.py

from Back.ml import ML

Импортирует основной класс ML-рекомендательной системы для доступа ко всем методам из других модулей.

ml = ML()

Создает глобальный singleton экземпляр ML-модели при загрузке модуля, обеспечивая единственный экземпляр с предобученными TF-IDF векторами.

Импорты

from transformers import AutoModelForCausalLM, AutoTokenizer

Импортирует основные классы HuggingFace Transformers для загрузки и работы с предобученными языковыми моделями, обеспечивая доступ к архитектуре CausalLM и токенизатору.

import torch

Предоставляет PyTorch тензоры, GPU-ускорение и функции инференса для выполнения нейронных вычислений языковой модели на аппаратном уровне.

import os

Обеспечивает операции с файловой системой: проверку существования файлов, создание директорий и работу с путями для кэширования модели.

from .cleaner import tclean

Импортирует функцию предобработки текста для очистки пользовательских запросов перед подачей в LLM (лемматизация, стоп-слова).

Класс LLM init(self)

self.local_dir = "stablelm_local"

Задаёт локальную директорию для кэширования весов модели (~8GB), избегая повторных загрузок с HuggingFace Hub.

self.flag_file = os.path.join(self.local_dir, "model_ready.flag")

Создаёт путь к флаговому файлу, который сигнализирует об успешной первой загрузке модели и готовности к использованию.

os.makedirs(self.local_dir, exist_ok=True)

Создаёт директорию кэша модели если она не существует, используя exist_ok для безопасности при повторных запусках.

self.model_name = "Qwen/Qwen3-4B-Instruct-2507"

Указывает конкретную инструкционную модель Qwen 4B параметров, оптимизированную для генерации текстовых ответов по системным промптам.

if not os.path.exists(self.flag_file):

Проверяет флаг первой загрузки, выполняя скачивание модели только при первом запуске, ускоряя последующие инициализации.

tokenizer = AutoTokenizer.from_pretrained(self.model_name)

Скачивает токенизатор модели с HuggingFace Hub для преобразования текста в числовые токены.

tokenizer.save_pretrained(self.local_dir)

Сохраняет токенизатор локально для быстрой загрузки без обращения к интернету при перезапусках.

model = AutoModelForCausalLM.from_pretrained(...)

local_files_only=False

Разрешает скачивание весов модели (4B параметров ~8GB) с HuggingFace Hub при первом запуске.

dtype=torch.bfloat16

Использует 16-битный формат bfloat16 для оптимизации памяти GPU (экономия ~50% VRAM по сравнению с float32).

device_map="auto"

Автоматически распределяет слои модели по доступным GPU/CPU, поддерживая работу на системах с разным количеством видеопамяти.

model.save_pretrained(self.local_dir)

Кэширует веса модели локально для мгновенной загрузки при последующих запусках (без интернета).

with open(self.flag_file, "w") as f: f.write("ok")

Создает флаг успешной загрузки, предотвращая повторное скачивание модели при перезапусках.

self.tokenizer = AutoTokenizer.from_pretrained(self.local_dir, local_files_only=True)

Загружает токенизатор из локального кэша для мгновенного старта без сетевых запросов.

**self.model = AutoModelForCausalLM.from_pretrained(self.local_dir,
local_files_only=False, dtype=torch.bfloat16, device_map="auto")**

Загружает модель из локального кэша в GPU-память с оптимизацией формата и автоматическим распределением.

describeGenres(self, usr_prompt: str)

prompt = ("Ты библиотекарь и литературный эксперт...")

Системный промпт задает роль библиотекаря с инструкциями по строгому формату ответа (список жанров без лишнего текста).

usr_prompt = tclean(usr_prompt)

Предобрабатывает пользовательский запрос: приводит к нижнему регистру, удаляет пунктуацию, лемматизирует, убирает стоп-слова.

full_prompt = f"{usr_prompt}\n\n{prompt}\n"

Конкатенирует очищенный запрос пользователя с системным промптом, разделяя пустыми строками для четкой структуры.

device = next(self.model.parameters()).device

Автоматически определяет устройство модели (cuda:0 или cpu) для корректной отправки тензоров.

inputs = self.tokenizer(full_prompt, return_tensors="pt").to(device)

Токенизирует полный промпт в PyTorch тензоры и переносит на GPU для инференса модели.

with torch.inference_mode():

Активирует режим инференса: отключает вычисление градиентов, значительно экономя память GPU (~30-50%).

****outputs = self.model.generate(inputs, max_new_tokens=20)**

Генерирует ответ LLM, ограничивая 20 новыми токенами для получения краткого списка жанров без длинных текстов.

generated_tokens = outputs[:, inputs["input_ids"].shape[-1]:]

Извлекает только сгенерированные токены, исключая входной промпт из результата генерации.

answer = self.tokenizer.decode(generated_tokens, skip_special_tokens=True)

Декодирует токены в читаемый текст, удаляя специальные символы (EOS, PAD) для чистого вывода жанров.

return answer

Возвращает строковое описание жанров (например: "фантастика детектив"), готовое для TF-IDF поиска похожих книг.

Основные библиотеки ML и NLP

accelerate==1.10.1

Оптимизирует загрузку и инференс LLM на GPU, автоматически распределяя слои модели по нескольким устройствам (device_map="auto") и управляя памятью для больших трансформеров.

nlTK==3.9.2

Библиотека обработки естественного языка, предоставляет токенизатор punkt и стоп-слова (закомментировано в cleaner.py), готова к расширению стандартного NLP-стека.

numpy==2.0.2

Основная библиотека численных вычислений, обеспечивает многомерные массивы и математические операции для работы с TF-IDF матрицами и векторами сходства.

pandas==2.3.3

Работа с табличными данными книг, преобразует SQLite таблицы и CSV файлы в DataFrame для эффективной фильтрации, группировки и подготовки данных ML.

pymorphy2==0.9.1

Морфологический анализатор русского языка, выполняет лемматизацию через morph.parse(t).normal_form() в функции tclean(), приводя слова к начальной форме.

pymorphy2-dicts-ru==2.4.417127.4579844

Морфологические словари для pymorphy2, содержит данные о формах русских слов, склонениях, спряжениях для точной лемматизации.

razdel==0.5.0

Специализированный токенизатор русского языка, корректно разбивает текст на слова с учетом дефисов, сокращений и особенностей морфологии (tokenize(text_cleaned)).

scikit-learn==1.6.1

Основной ML-фреймворк проекта: TfidfVectorizer для векторизации текстов, cosine_similarity для поиска похожих книг по TF-IDF.

PyTorch и Transformers (LLM)

torch==2.8.0

PyTorch фреймворк для нейронных сетей, обеспечивает model.generate(), bfloat16 тензоры, GPU-ускорение и inference_mode() для языковой модели.

transformers==4.57.0

HuggingFace библиотека для работы с трансформерами, предоставляет AutoModelForCausalLM и AutoTokenizer для загрузки Qwen модели.

tokenizers==0.22.1

Быстрый токенизатор Rust-based, используется в `self.tokenizer(full_prompt, return_tensors="pt")` для эффективной обработки длинных промптов.

huggingface-hub==0.35.3

Скачивает модели и токенизаторы с HuggingFace Hub через `from_pretrained("Qwen/Qwen3-4B-Instruct-2507")`, управляет репозиториями.

safetensors==0.6.2

Безопасная и быстрая загрузка весов моделей в формате `.safetensors`, избегает рисков `pickle` при загрузке трансформеров.

Базы данных и парсинг

sqlite3 (встроенная)

Локальная файловая база данных `DataBase.db` с таблицами `users/books/review`, обеспечивает постоянное хранение пользователей, книг и оценок.

requests==2.32.5

HTTP-клиент для парсинга `litres.ru`, выполняет GET-запросы к каталогам и страницам книг в `bookParse.py`.

beautifulsoup4==4.14.2

HTML-парсер для извлечения данных с `litres.ru`: `soup.find('h1')` для названий, `soup.select()` для карточек книг и описаний.

Вспомогательные библиотеки

tqdm==4.67.1

Отображает прогресс-бары при загрузке больших LLM-моделей (~8GB), показывает процент скачивания, скорость и оставшееся время для мониторинга.

filelock==3.19.1

Обеспечивает блокировку файлов при конкурентной загрузке моделей несколькими процессами, предотвращая конфликты при одновременном доступе к кэшу HuggingFace.

fsspec==2025.9.0

Универсальный файловый API для HuggingFace кэширования, поддерживает локальные файлы, HTTP, S3 для прозрачной работы с моделями из разных источников.

joblib==1.5.2

Параллелизует обучение `TfidfVectorizer` на многоядерных CPU, ускоряя `fit_transform()` для тысяч текстовых описаний книг.

scipy==1.13.1

Научные вычисления для оптимизации `cosine_similarity`, предоставляет разреженные матрицы и быстрые алгоритмы для больших TF-IDF векторов.

Telegram Bot (Aiogram)

aiogram (отсутствует в списке?)

Основная библиотека Telegram Bot API, реализует асинхронные Router handlers, InlineKeyboardMarkup, callback_query для интерактивного интерфейса рекомендаций.

Pillow==11.3.0

Обработка изображений обложек книг, преобразует локальные файлы в FSInputFile(photo) для отправки фото через Telegram Bot API с кнопками управления.

Визуализация и утилиты

matplotlib==3.9.4

Библиотека построения графиков (зависимость scikit-learn), содержит contourpy/fonttools для визуализации TF-IDF матриц при отладке.

PyYAML==6.0.3

Парсинг YAML-конфигурационных файлов, загрузка переменных окружения (токены бота, пути к БД) из config.yaml или .env.

Русский NLP стек

DAWG-Python==0.7.2

Directed Acyclic Word Graph для rumorphy2, обеспечивает быстрый поиск морфологических форм русских слов (миллионы запросов/сек).

docopt==0.6.2

Парсер аргументов командной строки для утилит (bookParse.py, Joiningtables.py), автоматически генерирует --help из docstring.

Файл stop_ru.txt

stop_ru.txt

Кастомный список русских стоп-слов для TF-IDF векторизации, содержит высокочастотные служебные слова, не несущие семантической нагрузки.

```
python
with open('Back/stop-ru.txt') as f: sw = f.read().splitlines()
self.vectorizer = TfidfVectorizer(stop_words=sw, max_features=20000)
```

Содержимое: Список часто встречающихся слов ("и", "в", "на", "не", "что", "как", "за", "на", "по", "от", ...) которые удаляются из текстов перед TF-IDF, повышая качество векторов рекомендаций.

CSV файлы данных книг

Combined_clean_tables.csv

Основная консолидированная база книг для инициализации ML-модели, содержит все данные из парсеров litres.ru (названия, авторы, жанры, описания, обложки, рейтинги), загружается в SQLite через cursor.executemany(INSERT INTO books...).

Lite.csv

Исходный легкий датасет книг (ограниченный объем данных), объединяется с другими

CSV в итоговый Combined_clean_tables.csv через скрипт Joiningtables.py для создания полной библиотеки.

Serious.csv

Исходный расширенный датасет книг (большой объем данных), объединяется с Lite.csv в итоговый Combined_clean_tables.csv через скрипт Joiningtables.py для создания полной библиотеки.

Модуль bookParse.py — Парсер litres.ru

Импорты

import requests

HTTP-клиент для выполнения GET-запросов к каталогам и страницам книг litres.ru, получает HTML-контент каталогов и карточек товаров.

from bs4 import BeautifulSoup

HTML-парсер для извлечения структурированных данных из страниц litres.ru: названия книг, авторов, описаний, жанров, обложек через soup.find_all() и soup.select().

from urllib.parse import urljoin

Преобразует относительные ссылки (href="/book/123") в абсолютные URL (<https://www.litres.ru/book/123>) для перехода на страницы конкретных книг.

import csv

Сохранение спарсенных данных книг в CSV-файлы с разделителем ';', writer.writerow() записывает каждую книгу как строку с полями.

Настройка

```
table_name = str(input("Введите название таблицы: "))
```

Получает от пользователя имя выходного CSV-файла (например: "Lite", "Serious"), определяет название файла для сохранения спарсенных данных.

```
**# with open(table_name + ".csv", "w",...)
```

Закомментированный блок создания CSV с заголовками, файл создается вручную без автоматических заголовков для совместимости с Joiningtables.py.

Параметры запроса

```
url = str(input("Введите ссылку на каталог: "))
```

Получает базовый URL каталога litres.ru от пользователя (например: <https://www.litres.ru/>), используется как основа для всех GET-запросов с пагинацией.

```
params = {"languages": "ru", "art_types": "text_book", "page": "0"}
```

Фильтрует каталог: только русскоязычные книги (languages=ru), текстовые книги (art_types=text_book), page управляет пагинацией.

```
headers = {"User-Agent": "Mozilla/5.0 ..."}"
```

Имитирует браузерный запрос, обходит защиту от ботов и парсеров, используя реалистичный User-Agent Chrome/Firefox.

```
pages = int(input("Введите кол-во страниц, которое нужно просмотреть: "))
```

Задаёт пользователем диапазон парсинга (количество страниц каталога), контролирует объём собираемых данных.

Основной цикл парсинга страниц

```
for page in range(486, pages+1):
```

Итерирует по страницам каталога начиная с 486-й (пропуск уже спарсенных данных), обеспечивает непрерывный сбор без дублирования.

```
params["page"] = str(page)
```

Обновляет параметр пагинации для текущей страницы каталога перед каждым HTTP-запросом.

```
response = requests.get(url, headers=headers, params=params)
```

Выполняет HTTP GET-запрос к текущей странице каталога `litres.ru` с фильтрами и заголовками браузера.

```
soup = BeautifulSoup(response.text, "html.parser")
```

Парсит полученный HTML в объект BeautifulSoup, создавая дерево DOM для поиска карточек книг и извлечения данных.

Сбор ссылок на книги

```
books_urls = []
```

Инициализирует пустой список для аккумуляции абсолютных URL всех книг на текущей странице каталога `litres.ru`.

```
divs = soup.find_all("div", class_=["baf48440", "b01c802a"])
```

Закомментированный альтернативный селектор карточек книг, не используется в активном коде парсера.

```
for div in soup.select('div[data-testid="art__wrapper"]):
```

Итерирует по всем контейнерам книг на странице используя современный CSS селектор `data-testid="art__wrapper"`.

```
a_tag = div.find("a", href=True)
```

Извлекает тег `<a>` с атрибутом `href` из каждой карточки книги, содержащий относительную ссылку на страницу книги.

```
full_url = urljoin("https://www.litres.ru", a_tag["href"])
```

Преобразует относительный href (`/book/123456`) в абсолютный URL (`https://www.litres.ru/book/123456`) для последующего парсинга.

Парсинг отдельной книги

for book in books_urls:

Проходит по всем собранным ссылкам книг, парсит каждую страницу индивидуально для извлечения детальной информации.

response = requests.get(book)

Выполняет HTTP GET-запрос к странице конкретной книги litres.ru для загрузки HTML-контента.

spans = soup.find_all("span")

Загружает все SPAN-элементы страницы для последующего поиска автора и даты публикации по индексам/тексту.

namesoup = soup.find("h1")

Находит главный заголовок H1 страницы, содержащий основное название книги.

if namesoup == None: continue

Пропускает некорректные страницы без названия (ошибки загрузки, редиректы), избегая сбоев парсера.

name = namesoup.getText()

Извлекает чистый текст названия книги из H1-заголовка без HTML-тегов.

author = spans.get_text(strip=True)[habr](#)

Хардкодированно берет автора из третьего SPAN-элемента страницы (структура litres.ru).

date = ''

Инициализирует пустую строку для хранения даты написания/публикации книги.

for i in range(1, len(spans)):

Последовательно сканирует все SPAN-элементы страницы в поисках маркера даты.

if spans[i].get_text(strip=True) == "Дата написания":

Ищет точное текстовое совпадение "Дата написания:" как индикатор поля с датой публикации.

date = spans[i+1].get_text()

Берет значение даты из следующего SPAN после маркера "Дата написания:".

discription = ''

Инициализирует пустую строку для аккумуляции полного текста описания книги.

dscrp = soup.find("div", class_ = ["ac83cc29"])

Находит контейнер описания книги по специфическому CSS-классу litres.ru.

for p in dscrp: discription += p.get_text()

Конкатенирует текст всех параграфов <p> внутри блока описания, собирая полное описание книги.

genres = soup.find("div", class_ = "aa63d864").get_text(strip = True)

Извлекает жанры книги из блока с CSS-классом "aa63d864", очищая от пробелов (strip=True).

```
pic = soup.find("div", class_ = ["_0bd8490c", "_78d816e0"])
```

Находит контейнер обложки по одному из двух возможных CSS-классов litres.ru.

```
pic = pic.find("img")
```

Внутри контейнера обложки ищет тег `<img>` с изображением.

```
pic = pic.get("src")
```

Извлекает URL обложки из атрибута `src` тега изображения.

```
scoresoup = soup.find("div", class_ = "_1419d759")
```

Находит блок рейтинга книги по уникальному CSS-классу litres.ru.

```
score = scoresoup.get_text()
```

Извлекает числовое значение рейтинга из блока оценки.

Сохранение в CSV

```
with open(table_name + '.csv', "a", newline="", encoding="utf-8-sig") as file:
```

Открывает CSV-файл в режиме дозаписи (append), добавляя новые книги в конец существующего файла. Использует UTF-8 с BOM для корректного отображения русских символов в Excel.

```
writer = csv.writer(file, delimiter=';')
```

Создает CSV-писатель с разделителем точка с запятой (';'), совместимый с европейскими локалями и скриптом `Joiningtables.py`.

```
writer.writerow([name, author, date, discription, genres, pic, book, score])
```

Записывает одну строку данных книги: название, автор, дата, описание, жанры, URL обложки, URL страницы, рейтинг — 8 полей в строго заданном порядке.

```
print("Отсканировано страниц: ", page)
```

Выводит прогресс парсинга в консоль, показывая номер успешно обработанной страницы каталога для мониторинга процесса.

Импорты

```
import pandas as pd
```

Обеспечивает чтение и обработку CSV-файлов с данными книг через `pd.read_csv()`, преобразует в `DataFrame` для удобного доступа по номеру строки.

```
import sqlite3
```

Предоставляет подключение к SQLite базе данных `DataBase.db` для загрузки объектов `Book` напрямую из таблиц `books/review`.

Класс Book init(self)

```
self.name = None
```

Инициализирует поле названия книги пустым значением, готовит объект для последующей загрузки данных из CSV или SQLite.

self.author = None

Поле для хранения имени автора книги, заполняется при загрузке из источника данных.

self.date = None

Хранит дату написания/издания книги, извлекается из CSV или базы данных.

self.discription = None

Поле аннотации/описания книги (опечатка: должно быть description), содержит текст с litres.ru.

self.genres = None

Список жанров книги в виде строки, используется для TF-IDF рекомендаций.

self.pic = None

URL изображения обложки книги для отправки в Telegram через FSInputFile.

self.url = None

Полный URL страницы книги на litres.ru для возможных ссылок пользователю.

self.score = None

Числовой рейтинг книги (например: 4.2), используется для сортировки рекомендаций.

**get_from_csv(self, line_number,
file_path="Data/Combined_clean_tables.csv")**

try:

Оборачивает весь процесс загрузки в блок обработки ошибок, перехватывает исключения CSV-парсинга и некорректных номеров строк.

df = pd.read_csv(file_path, sep=';', encoding='utf-8')

Загружает основной CSV-файл Combined_clean_tables.csv в Pandas DataFrame с разделителем ';' и UTF-8 кодировкой.

if line_number < 1 or line_number > len(df):

Проверяет валидность номера строки, исключая нулевые/отрицательные значения и превышение общего количества книг.

print(f"Строка {line_number} вне диапазона (1-{len(df)})")

Выводит информативное сообщение об ошибке с указанием допустимого диапазона строк для пользователя.

data_index = line_number - 1

Преобразует номер строки из 1-based нотации (человекочитаемый) в 0-based индексацию Pandas DataFrame.

line = df.iloc[data_index]

Извлекает конкретную строку DataFrame по индексу, возвращая объект Series с данными книги.

self.name = line.get('name')

Безопасно заполняет поле названия книги методом `get()`, возвращает `None` при отсутствии столбца.

self.author = line.get('author') / self.date = line.get('date') и т.д.

Последовательно заполняет все поля объекта `Book` из соответствующих столбцов CSV-строки с безопасным извлечением.

print(f'Книга '{self.name}' загружена из строки {line_number}')

Подтверждает успешную загрузку конкретной книги с указанием названия и номера строки для логирования.

except Exception as e: print(f'Ошибка загрузки: {e}')

Перехватывает любые исключения (файл не найден, поврежденный CSV, кодировка) и выводит понятное сообщение об ошибке.

display_info(self)

if self.name:

Проверяет успешную загрузку книги по наличию названия, избегает вывода информации о пустых объектах.

print('\n' + f'{self.name} | {self.author} | {self.date} | Score: {self.score}' + '\n')

Выводит основную информацию книги в компактном формате: название | автор | дата | рейтинг для быстрого обзора.

print(f'Description: {self.description}' + '\n')

Отображает полное описание/аннотацию книги для ознакомления с содержанием.

print(f'Genres: {self.genres}' + '\n')

Показывает список жанров книги, используемых для TF-IDF рекомендаций.

print(f'pic: {self.pic}' + '\n')

Выводит URL обложки книги для проверки доступности изображения.

if self.url != None: print(f'URL: {self.url}' + '\n')

Опционально показывает ссылку на `litres.ru` только при наличии URL (отсутствует в SQLite).

else: print("Книга не загружена")

Информирует о пустом объекте `Book` если данные не были загружены.

if self.name != None:

Проверяет наличие данных книги перед формированием кортежа для SQL-вставки.

return (self.name, self.author, self.date, self.genres, self.description, self.pic, self.score)

Возвращает кортеж из 7 полей для `cursor.executemany("INSERT INTO books ...")`, совместимый со схемой таблицы `books`.

db = sqlite3.connect(file_path)

Подключается к базе данных для загрузки книги по первичному ключу ID.

cursor.execute("SELECT * FROM books WHERE id = ?", (id,))

Выполняет точный поиск книги по ID с параметризованным запросом для безопасности.

b = cursor.fetchone()

Получает единственную строку результата (или None если книга не найдена).

if b != None:

Проверяет успешный поиск книги перед заполнением полей объекта.

self.id = id

Сохраняет ID книги как атрибут объекта для последующих операций (findReview, review).

self.name = b / self.author = b и т.д.

Маппит поля результата запроса на атрибуты объекта: b=name, b=author, b=date.

db.close()

Освобождает ресурсы базы данных после загрузки книги.

book = Book()

Создает новый пустой объект Book для фабричного метода загрузки по ID.

book.url = None

Явно сбрасывает поле URL (отсутствует в схеме SQLite таблицы books).

book.get_from_sql(bd_id, file_path)

Заполняет объект данными книги из базы данных по указанному ID.

return book

Возвращает готовый заполненный объект Book для использования в рекомендациях.

**cursor.execute("SELECT esteem FROM review WHERE idBook = ? AND idUser = ?",
(self.id, idUser))**

Ищет отзыв пользователя для конкретной книги по паре (idBook, idUser) в таблице review.

b = cursor.fetchone()

Получает оценку (esteem: 0-3) или None если отзыва нет.

if b != None: return int(b)

Возвращает числовую оценку: 0=просмотрено, 1=лайк, 2=дизлайк, 3=сохранено.

else: return None

Сигнализирует о первом просмотре книги пользователем (отсутствует запись в review).

if user_input < 0 or user_input > 3: user_input = 0

Валидирует пользовательский ввод, устанавливая 0 (просмотрено) при некорректных значениях.

reviewExist = self.findReview(id_user)

Проверяет наличие существующего отзыва для определения INSERT или UPDATE.

db.execute("PRAGMA foreign_keys = ON")

Активирует внешние ключи с каскадным удалением для целостности таблиц.

if reviewExist == None:

Новый отзыв (первый просмотр/оценка) — выполняет INSERT в таблицу review.

cursor.execute("INSERT INTO review (idBook, idUser, esteem) VALUES (?, ?, ?)", ...)

Добавляет новую запись отзыва с внешними ключами на books и users.

else:

Существующий отзыв — обновляет поле esteem существующей записи.

cursor.execute("UPDATE review SET esteem = ? WHERE idBook = ? AND idUser = ?", ...)

Изменяет оценку конкретной книги для конкретного пользователя.

db.commit() / db.close()

Фиксирует изменения в базе данных и освобождает соединение.

Таблица users

CREATE TABLE IF NOT EXISTS users

Создает таблицу пользователей Telegram если она не существует, обеспечивает идемпотентность скрипта инициализации.

(

Открывает блок определения колонок таблицы users.

id INTEGER PRIMARY KEY

Уникальный Telegram userId как первичный ключ, напрямую используется из Telegram API без преобразований.

indx INTEGER

Индекс текущей книги в рекомендациях пользователя, отслеживает позицию навигации (не используется в текущей версии).

currentCategory TEXT

Текущая категория рекомендаций ("likes"/"dislikes"/"bookmarks"), поддерживает переключение между видами контента.

source TEXT

Источник текущих рекомендаций ("search"/"ratings"/"description"), определяет алгоритм подбора книг.

favAuthor TEXT

Предпочитаемый автор пользователя для персонализации (не используется в текущей версии системы).

favGenre TEXT

Предпочитаемый жанр пользователя для персонализации (не используется в текущей версии системы).

);

Завершает определение таблицы users.

Таблица books

CREATE TABLE IF NOT EXISTS books

Создает основную таблицу библиотеки книг если не существует, хранит все спарсенные данные litres.ru.

(

Открывает блок колонок таблицы books.

id INTEGER PRIMARY KEY AUTOINCREMENT

Автоинкрементный первичный ключ, автоматически присваивает уникальные ID каждой загруженной книге (1,2,3...).

name TEXT NOT NULL

Обязательное название книги, ключевое поле для поиска и рекомендаций (например: "Война и мир").

author TEXT

Имя автора книги, может быть NULL для коллективных изданий или анонимных работ.

genre TEXT

Строка жанров в формате "фантастика,детектив,боевик", используется для TF-IDF векторизации.

year INTEGER

Год написания/издания книги из парсера litres.ru.

discription TEXT

Полное описание/аннотация книги для информирования пользователя (опечатка: description).

picPath TEXT

URL изображения обложки для отправки в Telegram Bot через FSInputFile.

score FLOAT

Числовой рейтинг книги (4.2, 4.8), используется для сортировки популярных рекомендаций.

);

Завершает определение таблицы books.

Таблица review

CREATE TABLE IF NOT EXISTS review

Создает связующую таблицу отзывов пользователей о книгах, реализует many-to-many связь users↔books.

(

Открывает структуру таблицы review.

id INTEGER PRIMARY KEY AUTOINCREMENT

Автонумерация записей отзывов для внутреннего использования.

idBook INTEGER NOT NULL

Обязательная внешняя ссылка на ID книги из таблицы books.

idUser INTEGER NOT NULL

Обязательная внешняя ссылка на ID пользователя из таблицы users.

esteem INTEGER NOT NULL

Тип оценки: 0=просмотрено, 1=лайк, 2=дизлайк, 3=сохранено — основа персонализации.

-- 0 - seen, 1 - like, 2 - dislike, 3 - saved

Документация кодов оценок для разработчиков.

FOREIGN KEY (idBook) REFERENCES books(id) ON DELETE CASCADE

Внешний ключ на таблицу books с каскадным удалением: при удалении книги удаляются все отзывы.

FOREIGN KEY (idUser) REFERENCES users(id) ON DELETE CASCADE

Внешний ключ на таблицу users с каскадным удалением: при удалении пользователя удаляются все его отзывы.

Завершает схему базы данных с полной референциальной целостностью.

Файл Genres.txt

Genres.txt

Содержит уникальный отсортированный список всех жанров из CSV-файлов (по одному жанру на строку), используется как справочный словарь для LLM-промпта в model.py.

Закомментировано: # with open(genres_path) as f: genres = f.read().

Импорты

import pandas as pd

Обеспечивает обработку CSV-таблиц: pd.read_csv() для загрузки, pd.concat() для объединения, drop_duplicates() для удаления повторов.

Настройка файлов

file_list = ["Lite.csv", "Serious.csv"]

Определяет список исходных CSV-датасетов от парсера bookParse.py для автоматического объединения в единую библиотеку.

cleaned_tables = []

Инициализирует пустой список для аккумуляции очищенных DataFrame из каждого успешно обработанного CSV-файла.

all_genres = set()

Создает множество для сбора уникальных жанров из всех книг (не используется в активном коде, закомментировано).

Обработка файлов

for file in file_list:

Итерирует по всем исходным CSV-файлам (Lite.csv + Serious.csv), обрабатывая каждый отдельно.

try:

Оборачивает обработку каждого файла в блок исключений, обеспечивая продолжение работы при ошибке одного датасета.

df = pd.read_csv(file, sep=';', encoding='utf-8', dtype=str)

Загружает CSV с разделителем ';' (из bookParse.py), UTF-8 кодировкой, все поля как строки для сохранения формата.

df_clean = df.drop_duplicates()

Удаляет дублирующиеся строки внутри каждого CSV-файла, сохраняя только уникальные книги.

cleaned_tables.append(df_clean)

Добавляет очищенную таблицу в аккумулятор для последующего объединения.

print(f"Обработан файл: {file}")

Подтверждает успешную обработку каждого файла в консоли для мониторинга прогресса.

except Exception as e: print(f"Ошибка при обработке файла {file}: {e}")

Перехватывает ошибки чтения/парсинга, выводит сообщение и продолжает обработку остальных файлов.

Объединение таблиц

if cleaned_tables:

Проверяет наличие успешно обработанных таблиц перед объединением, избегает ошибок на пустом списке.

combined_table = pd.concat(cleaned_tables, ignore_index=True)

Вертикально объединяет все DataFrame (Lite + Serious) в одну таблицу, сбрасывая индексы для непрерывной нумерации.

combined_table = combined_table.drop_duplicates()

Выполняет финальную дедупликацию по всем полям, удаляя книги, повторяющиеся между разными CSV-файлами.

output_file = "Combined_clean_tables.csv"

Задаёт имя основного выходного файла, используемого ML-моделью для инициализации базы данных.

combined_table.to_csv(output_file, index=False, sep=';', encoding='utf-8')

Сохраняет объединённую таблицу без индексов, с разделителем ';', UTF-8 для совместимости с book.py и ml.py.

print(f"Объединённая таблица сохранена в: {output_file}")

Подтверждает успешное создание основного датасета для системы рекомендаций.

Извлечение жанров

for genres_str in combined_table['genres'].dropna():

Итерирует по всем непустым значениям столбца 'genres' объединённой таблицы, исключая NaN/NULL с помощью dropna().

genres_list = [genre.strip() for genre in genres_str.split(', ')]

Разбирает строку жанров ("фантастика, детектив, боевик") на список, удаляя пробелы по краям каждого жанра через strip().

all_genres.update(genres_list)

Добавляет все жанры из текущей книги в множество уникальных жанров, автоматически исключая дубликаты.

if all_genres:

Проверяет наличие собранных жанров перед созданием файла, избегает записи пустого genres.txt.

with open('genres.txt', 'w', encoding='utf-8') as f:

Создаёт/перезаписывает файл genres.txt в режиме записи с UTF-8 кодировкой для русских названий жанров.

for genre in sorted(all_genres): f.write(genre + '\n')

Записывает все уникальные жанры в алфавитном порядке, по одному на строку для удобства чтения в LLM.

print(f"Уникальные жанры сохранены в файл genres.txt, всего их: {len(all_genres)}")

Выводит статистику: общее количество уникальных жанров (например: 1500) для контроля качества парсинга.

Обработка ошибок

else: print("Жанры не найдены ни в одном файле")

Информирует если столбец 'genres' пустой во всех книгах объединённой таблицы (ошибка парсера bookParse.py).

else: print("Ни один файл не был успешно обработан")

Сигнализирует о полном провале: ни один CSV-файл (Lite.csv/Serious.csv) не удалось загрузить или обработать.

Импорты

from aiogram import types, Router, F

Импортирует ядро Aiogram: types.Message для обработки сообщений, Router для регистрации обработчиков, F для фильтров callback_data.

from aiogram.types import InlineKeyboardButton, InlineKeyboardMarkup

Предоставляет inline-клавиатуры Telegram: кнопки с callback_data для навигации, лайков, дизлайков без отправки сообщений.

from aiogram.types.input_file import FSInputFile

Отправка локальных изображений обложек книг в Telegram через photo=FSInputFile(picPath) (закомментировано в коде).

from Front.test_books import books

Тестовый список книг для отладки (не используется в продакшене, закомментировано).

from Front.keyboards.main_menu import main_menu

Главное меню бота: кнопки "Подборка по жанрам", "Мои оценки", "Поиск книг".

from Back.ml_object import ml

Глобальный экземпляр ML-модели: ml.recommendations(), ml.GenreCossim(), ml.NameCossim() для генерации рекомендаций.

from Data.book_for_tgBot import Book

Класс Book для загрузки книг по ID: Book.byID(book_id) возвращает объект с name/author/genre/picPath.

from Back.ml import get_list_reviews, add_user_to_db

Базовые функции: get_list_reviews(user_id, esteem) для истории отзывов, add_user_to_db() для регистрации.

Router и состояние

router = Router()

Создает маршрутизатор Aiogram для регистрации всех callback_query и message handlers Telegram-бота.

user_feedback = {}

Глобальный словарь состояния пользователей: {user_id: {"likes": [], "dislikes": [], "bookmarks": [], "index": 0, "recommendations": [], "source": "ratings"}}.

Инициализация пользователя

def init_user(user_id):

Создает состояние нового пользователя при первом обращении к боту, регистрирует в БД и подготавливает рекомендации.

add_user_to_db(user_id)

Регистрирует Telegram user_id в таблице users, создает запись если не существует.

user_feedback[user_id] = {...}

Инициализирует словарь состояния: пустые списки лайков/дизлайков/закладок, index=0, пустые рекомендации, source="ratings".

"recommendations": []#ml.recommendations(user_id)

Закомментированная предварительная генерация рекомендаций при регистрации (заполняется по запросу).

Клавиатуры

book_keyboard() (строки 32-45)

Создает основную inline-клавиатуру для взаимодействия с книгой: 👍 (лайк), 👎 (дизлайк), 💾 (сохранить), ⬅️ (предыдущая), ➡️ (следующая), 🏠 (главное меню).

book_keyboard_in_list() (строки 47-55)

Клавиатура для списков/поиска: 👍 💾 👎 ⬅️ — лайк/сохранить/дизлайк без перехода к следующей книге, только "Назад".

ratings_menu_all() (строки 57-66)

Главное меню оценок: ❤️ (Мои лайки), 💔 (Дизлайки), 💾 (Закладки) — навигация по истории пользователя.

ratings_menu_category() / search_menu() (строки 68-85)

Клавиатуры навигации: "⬅️ К списку оценок"/"⬅️ В главное меню" — возвраты из подменю поиска и категорий.

Assistant: <|control12|>## LLM поиск по описанию

async def llm_send_book(message, user_id, edit=False):

Отправляет книгу по результатам LLM-поиска: ml.GenreCossim(text) → фото обложки + описание + клавиатура, поддерживает edit_media() для обновления.

books = ml.GenreCossim(text)

Преобразует текстовый запрос пользователя ("хочу фантастику как у Стругацких") в список ID книг через LLM + TF-IDF жанров.

user_feedback[user_id]["recommendations"] = books

Сохраняет список рекомендаций в состоянии пользователя для навигации ⬅️ ➡️ между книгами.

caption = f"{book.name}\n{book.description[:500]}"

Формирует HTML-карточку книги: жирное название + усеченное описание (500 символов) для Telegram.

message.edit_media()/answer_photo()

Обновляет существующее сообщение (edit_media) или отправляет новое фото с обложкой, caption и inline-клавиатурой.

Callback handlers

by_description_handler (строки 114-124)

Обработчик кнопки "🔍 По описанию": запрашивает текст через message.reply("Введите описание желаемых книг").

send_book() (строки 126-140) + selection_handler (строки 142-149)

Генерирует персональные рекомендации ml.recommendations(user_id), сохраняет в состояние, отправляет первую книгу.

give_feedback() (строки 151-158) + feedback_handler (строки 160-181)

Сохраняет оценку book.review(user_id, 1/2/3) в таблицу review, автоматически переходит к следующей книге: index = (index + 1) % len(books).

nav_handler (строки 183-194)

Обеспечивает кольцевую навигацию: (index + 1/-1) % len(recommendations) — конец списка возвращает к началу.

back_to_menu (строки 196-200)

Возвращает пользователя в главное меню через main_menu() клавиатуру.

Мои оценки

my_ratings_handler

Обработчик кнопки "Мои оценки": показывает меню ❤️ (лайки) 💔 (дизлайки)

🔖 (закладки) с историей взаимодействий.

get_list_reviews(user_id, 1/2/3)

Извлекает ID книг по типу оценки: 1=лайки, 2=дизлайки, 3=закладки из таблицы review.

f"{i+1}. {Book.byID(id).name}"

Формирует нумерованный список книг: "1. Война и мир\n2. 1984" для текстового сообщения.

show_category_handler (строки 229-254)

Парсит callback_data "show_likes" → current_category="likes", показывает книги категории.

data["current_category"] = callback_query.data.split("_")

Разбирает callback_data: "show_likes" → "likes", "show_dislikes" → "dislikes".

"📖 Введите номер книги"

Запрашивает цифру для интерактивного выбора книги из списка оценок (1, 2, 3 → Book.byID(id)).

Поиск книг

start_search_handler

Обработчик кнопки "🔍 Поиск": запрашивает название книги через message.reply("Введите название для поиска").

Обработчик текста (универсальный)

@router.message(lambda message: True)

Универсальный обработчик всех текстовых сообщений бота, определяет контекст по состоянию user_feedback[user_id].

Обработка цифр

if text.isdigit() and data.get("current_index_in_list") is None:

Обрабатывает числовой ввод только в режимах поиска/оценок, игнорирует цифры в других контекстах.

idx = int(text) - 1

Преобразует ввод пользователя "1" → индекс 0 для выбора книги из нумерованного списка.

Поиск по названию

if data.get("source") == "search":

Проверяет режим поиска по названию, активированный кнопкой "Поиск".

ml.search(query)

Выполняет полнотекстовый поиск книг по названию через ml.search(): SQL LIKE '%query%' или линейный поиск.

Поиск в оценках

results = [b for b in all_books if query in b.name.lower()]

Линейно фильтрует текущий список книг (лайки/дизлайки/закладки) по подстроке в названии, регистронезависимо.

LLM описание

if data.get("source") == "description_search":

Проверяет режим поиска по описанию жанров, активированный кнопкой "По описанию".

llm_send_book()

Отправляет первую книгу из LLM-рекомендаций ml.GenreCossim(query) с фото и клавиатурой.

Навигация назад

back_from_book_handler (строки 351-376)

Возвращает из просмотра карточки книги в список: сбрасывает `current_index_in_list = None`.

back_to_all_ratings_handler (строки 378-401)

Сбрасывает фильтры категорий: `source = None`, `current_category = None` → показывает все оценки сразу.

back_to_main_handler / back_to_main_from_search_handler (строки 403-434)

Полный сброс поиска: очищает `search_results = []`, возвращает в главное меню `main_menu()`.

feedback_in_list_handler (строки 436-460)

Обработчик оценок в списках: `callback_data="like_no_next"` — лайк/дизлайк/закладка без автоматического перехода к следующей книге.

Модуль `start.py`

Импорты

from aiogram import types, Router

Импортирует ядро Aiogram: `types.Message` для обработки входящих сообщений, `Router` для регистрации обработчиков команд.

from aiogram.filters import Command

Фильтр команд Telegram: `Command("start")` активирует обработчик только при вводе `/start`.

from Front.keyboards.main_menu import main_menu

Импортирует главную inline-клавиатуру бота с 4 кнопками навигации.

Router

router = Router()

Создает маршрутизатор Aiogram исключительно для обработки команды `/start`, регистрируется в `main.py`.

Обработчик команды `/start`

@router.message(Command("start"))

Декоратор фильтрует только сообщения с командой `/start`, игнорирует остальные входящие команды.

async def start_handler(message: types.Message):

Асинхронный обработчик первой команды пользователя, инициализирует взаимодействие с ботом.

```
welcome_text = "Привет, я бот библиотекарь\n❤️❤️❤️"
```

Текст приветствия с эмодзи сердечек, создает дружелюбный тон общения.

```
await message.answer(welcome_text, reply_markup=main_menu())
```

Отправляет приветственное сообщение с прикрепленной главной inline-клавиатурой (4 кнопки).

Модуль main_menu.py (keyboards)

Импорт

```
from aiogram.types import InlineKeyboardButton, InlineKeyboardMarkup
```

Импортирует классы для создания интерактивных кнопок под сообщением без отправки новых сообщений.

Функция main_menu()

```
def main_menu():
```

Возвращает готовую InlineKeyboardMarkup — центральный хаб навигации бота.

```
    InlineKeyboardMarkup(inline_keyboard=[...])
```

Создает вертикальное меню из 4 кнопок в одном столбце под сообщением.

```
    InlineKeyboardButton(text="Подборка", callback_data="selection")
```

Кнопка ML-рекомендаций: callback_data="selection" → selection_handler → ml.recommendations(user_id).

```
    InlineKeyboardButton(text="Мои оценки", callback_data="my_ratings")
```

Кнопка истории отзывов: callback_data="my_ratings" → my_ratings_handler → get_list_reviews().

```
    InlineKeyboardButton(text="Подборка по описанию", callback_data="by_description")
```

Кнопка LLM-поиска: callback_data="by_description" → by_description_handler → ml.GenreCossim().

```
    InlineKeyboardButton(text="Поиск", callback_data="search_books")
```

Кнопка поиска по названию: callback_data="search_books" → start_search_handler → ml.search().

```
    return InlineKeyboardMarkup(...)
```

Возвращает готовую клавиатуру для использования в /start, back_to_main_handler и других точках возврата.

Модуль Bot/main.py

Импорты

import asyncio

Обеспечивает асинхронный запуск бота: `asyncio.run(main())` для обработки множества одновременных запросов Telegram.

import logging

Настраивает логирование работы бота: INFO/ERROR сообщения с временными метками для мониторинга и отладки.

from aiogram import Bot, Dispatcher

Ядро Aiogram: `Bot(token)` для подключения к Telegram API, `Dispatcher()` для маршрутизации событий.

from dotenv import load_dotenv

Загружает переменные окружения из `.env` файлов, скрывая `BOT_TOKEN` от Git-репозитория.

import os

Доступ к системным переменным окружения: `os.getenv("BOT_TOKEN")` для безопасного чтения токена.

from Front.handlers.start import router as start_router

Импортирует роутер команды `/start`: приветствие + главное меню при первом запуске.

from Front.handlers.book_handlers import router as book_router

Импортирует основной роутер: все `callback_query` и `message handlers` логики рекомендаций.

Настройка логирования

logging.basicConfig(level=logging.INFO, ...)

Настраивает формат логов: "2025-12-04 23:51:23 - main - INFO - Бот запущен" с временем, модулем, уровнем.

logger = logging.getLogger(name)

Создает логгер для текущего модуля `main.py`: `logger.info()/error()` для структурированных сообщений.

Главная функция main()

async def main():

Точка входа полного жизненного цикла бота: инициализация → запуск polling → graceful shutdown.

load_dotenv()

Загружает `.env` файл из корня проекта, ищет `BOT_TOKEN` в переменных окружения.

load_dotenv('Front/config.env')

Резервная загрузка `config.env` из папки `Front` если основной `.env` отсутствует.

BOT_TOKEN = os.getenv("BOT_TOKEN")

Извлекает строку токена "8382913971:AAH..." из переменных окружения.

if not BOT_TOKEN: logger.error("Токен бота не найден!...")

Останавливает запуск с критической ошибкой если токен не найден в .env файлах.

bot = Bot(token=BOT_TOKEN)

Инициализирует подключение к Telegram Bot API с полученным токеном.

dp = Dispatcher()

Создает диспетчер событий для регистрации всех роутеров обработчиков.

dp.include_router(start_router)

Регистрирует роутер /start для обработки команды запуска бота.

dp.include_router(book_router)

Регистрирует основной роутер со всей логикой рекомендаций, поиска, отзывов.

logger.info("Бот запущен и готов к работе!")

Подтверждает успешную инициализацию всех компонентов перед polling.

await dp.start_polling(bot, allowed_updates=dp.resolve_used_update_types())

Запускает бесконечный long polling: опрос Telegram API на новые сообщения/callbacks.

finally: await bot.session.close()

Graceful shutdown: корректно закрывает HTTP-сессию бота при остановке.

Точка входа

if name == "main":

Условный запуск только при прямом вызове python Bot/main.py (не при импорте).

asyncio.run(main())

Асинхронно запускает главную функцию бота.

except KeyboardInterrupt:

Перехватывает Ctrl+C для корректной остановки: "Бот остановлен пользователем".

except Exception as e: logger.error(f"Произошла ошибка: {e}")

Логирует любые непредвиденные ошибки краша с полной трассировкой.

Файл config.env

BOT_TOKEN=8382913971:AAHvMPjombCtHLp2li0fJ8Ixr-vUQxqDk9E

Секретный токен Telegram бота в формате bot_id:api_hash (59 символов).

text

load_dotenv('Front/config.env') → os.getenv("BOT_TOKEN") → Bot(token)

Формат: bot_id:api_hash — никогда не публиковать в Git, хранить только в .env файлах!

Модуль Test_books.py

books = [{"id": 1, "title": "1984", "author": "Джордж Оруэлл", ...}]

Глобальный список из 10 классических книг для тестирования интерфейса бота без подключения к базе данных.

{"id": 1, "title": "1984"}

Классическая антиутопия Джорджа Оруэлла о тоталитаризме, Большом Брате, контроле мыслей.

{"id": 2, "title": "Мастер и Маргарита"}

Русская классика Булгакова: сатира, мистика, любовь, Воланд в Москве 1930-х.

{"id": 3, "title": "Преступление и наказание"}

Философский роман Достоевского: Раскольников, теория "твари дрожащей", совесть.

{"id": 4, "title": "Война и мир"}

Эпопея Толстого: Наполеоновские войны, русская душа, Пьер Безухов, Наташа Ростова.

{"id": 5, "title": "Гарри Поттер..."}

Современная фэнтези Дж. Р. Роулинг: Хогвартс, магия, борьба с Волдемортом.

{"id": 6, "title": "Убить пересмешника"}

Социальная драма Харпер Ли: расизм, юг США 1930-х, суд над невиновным.

{"id": 7, "title": "Алиса в Стране чудес"}

Детская классика Льюиса Кэрролла: абсурдная логика, Белый Кролик, Безумное чаепитие.

{"id": 8, "title": "Пикник на обочине"}

Русская научная фантастика Стругацких: Сталкер, Зона, артефакты пришельцев.

{"id": 9, "title": "Три товарища"}

Ремарк о послевоенной Германии: дружба, любовь, Роберт Локамп, инфляция 1920-х.

{"id": 10, "title": "Шерлок Холмс..."}

Первый кейс Конан Дойла: "Этюд в багровых тонах", Холмс встречается Ватсона.

 **Примечание:** Импортируется в book_handlers.py но не используется (заменено Book.byID() из реальной БД).

Файл .gitignore

Back/stablelm_local

Исключает локальный кэш LLM-модели Qwen3-4B-Instruct (~8GB весов + токенизатор) из Git-репозитория.

stablelm_local

Дублирующий паттерн для альтернативных путей кэша HuggingFace трансформеров.

venv/

Виртуальное окружение Python полностью исключается, зависимости фиксируются в requirements.txt.

Data/DataBase.db

Локальная SQLite база данных с книгами/пользователями/отзывами (создается при первом запуске ML).

covers/

Папка с изображениями обложек книг (covers/1984.jpg, covers/war_and_peace.jpg) для тестирования.

***pycache/, Data/pycache/, .рус**

Python bytecode кэш: автоматически генерируется при импорте, не требует версионирования.

*****.bin, .pt, .ckpt**

PyTorch чекпоинты и веса моделей: torch.save() файлы ML-моделей исключаются из Git.

.DS_Store

macOS системные метаданные файлов/папок, создаются Finder'ом на Mac.

***.log**

Логи работы бота (logging.INFO/ERROR), накапливаются при работе main.py.